Gujarat Police Hackathon

Git / GitHub Setup, Commands, Troubleshooting and Repository Cleanup

Document series: Part 3 - GitHub. This document follows the MediaMTX + FFmpeg and Docker documentation and records how the working project was connected to GitHub, how changes were committed, and how oversized local binaries were prevented from blocking the push.

Repository: honeybadger-REX/GUJARAT_POLICE_HACKETHON

1. Why Git and GitHub Were Used

Git was used to track source-code changes locally. GitHub was used as the remote repository where the project source and deployment configuration are stored. The important goal for this project was to keep source code and configuration portable while keeping machine-specific runtime binaries out of the repository.

```
Local project
    |
    | git commit
    v
Local Git history
    |
    | git push
    v
GitHub repository
honeybadger-REX/GUJARAT_POLICE_HACKETHON
```

2. Local Repository and Remote

The project was already a Git working copy, so there was no need to run git init or clone another copy. The existing remote was the GitHub project repository.

```
git remote -v

origin  https://github.com/honeybadger-REX/GUJARAT_POLICE_HACKETHON.git (fetch)
origin  https://github.com/honeybadger-REX/GUJARAT_POLICE_HACKETHON.git (push)
```

The remote points to the intended repository. This means normal commands such as **git push** can publish the local main branch to that repository.

3. Git Identity vs GitHub Authentication

Two GitHub accounts were available on the machine. The local Git commit identity was checked with git config. It was set to the project identity.

```
git config user.name
git config user.email

honeybadger-REX
[project Git email]
```

This identifies the author information stored in commits. It does **not** itself determine which GitHub account is authenticated for a push. Remote authentication is a separate concern.

GitHub CLI was checked with **gh auth status**, but gh was not installed. This was not required for the repository because normal Git remote authentication could still be used.

4. Basic Git Workflow Used

```
# See what changed
git status

# Review changes before staging
git diff

# Stage changes
git add .
```

```
# Create a commit
git commit -m "Fix camera streaming and exclude local FFmpeg binaries"

# Push main to GitHub
git push origin main
```

Recommended habit: run git status and git diff before git add . when large or machine-specific files may exist in the working directory.

5. First GitHub Push Problem - Files Over 100 MB

The initial push was rejected by GitHub because the local project included very large FFmpeg executable files. GitHub reported that ffmpeg.exe, ffplay.exe and ffprobe.exe were each larger than the 100 MB individual-file limit.

```
ffplay.exe   ~158 MB
ffmpeg.exe   ~156 MB
ffprobe.exe  ~156 MB

GitHub:
GH001: Large files detected
pre-receive hook declined
```

The project did not need these Windows binaries in Git because FFmpeg was being used as a local development/testing tool and the portable Docker design was intended to obtain runtime dependencies separately.

6. Fix - Keep Large Local Binaries Out of Git

The local FFmpeg directory was removed from Git tracking while intentionally keeping the files on the computer.

```
git rm -r --cached "ai/ffmpeg-master-latest-win64-gpl"
```

The **--cached** option removes the files from Git's index without deleting the local files from disk.

The same principle was used for the local Windows MediaMTX executable, because GitHub also warned that it was above the recommended 50 MB size.

```
git rm --cached "streaming/mediamtx_v1.21.0_windows_amd64/mediamtx.exe"
```

7. .gitignore Changes

To prevent the large local binaries from being accidentally re-added, the project .gitignore was updated.

```
# Local FFmpeg binaries
ai/ffmpeg-master-latest-win64-gpl/

# Local Windows MediaMTX executable
streaming/mediamtx_v1.21.0_windows_amd64/mediamtx.exe

# Python virtual environment
.venv/

# Python generated files
__pycache__/
*.pyc
```

The repository should contain source code, Dockerfiles, Compose configuration and text configuration files. It should not depend on a developer's local virtual environment or Windows executable folders.

8. Second GitHub Push Problem - Old Commits Still Contained FFmpeg

After removing FFmpeg from the current working tree, the push was still rejected. The reason was that the oversized files existed in earlier local commits that had not yet been pushed to GitHub.

```
origin/main
   |
   +-- local commit 1
   +-- local commit 2
   +-- local commit 3
   +-- local commit 4
             ^
             HEAD
```

```
The old commits still contained the large binaries.
```

Simply deleting the binaries in a new commit does not erase them from previous commits. The unpushed history therefore had to be rebuilt.

9. Rebuilding the Unpushed History Safely

Before rewriting local history, a backup branch was created.

```
git branch backup-before-cleanup
```

Then the local branch was moved back to the remote commit while preserving the local changes as staged changes.

```
git reset --soft origin/main
```

A soft reset moves HEAD but leaves the project changes available for a new clean commit. This was useful because the remote branch still pointed to the last accepted commit.

After the reset, the large binaries were excluded using .gitignore and Git's cached removal. A clean commit could then be created from the remaining project changes.

10. A Second Git Problem Found: .venv Accidentally Staged

During the cleanup, git status showed that the entire local Python virtual environment had become staged. The output included many files under **.venv/Lib/site-packages/** and even rename detections involving files under streaming. This was another sign that the environment should never be committed.

```
git status

Changes to be committed:
  modified: .gitignore
  new file: .venv/Lib/site-packages/...
  renamed: streaming/ffmpeg/README.md -> .venv/...
  renamed: streaming/mediamtx/mediamtx.yml -> .venv/...
```

The fix was to make sure .venv/ was ignored and not staged. This kept the repository focused on actual source/configuration rather than the local Python environment.

11. Useful Tracking Checks

```
# Check whether FFmpeg / MediaMTX binaries are still tracked
git ls-files | Select-String "ffmpeg|mediamtx.exe"

# If the command prints nothing:
# those specific files are not tracked in the current index.
```

The check above only describes the current tracked snapshot. It does not by itself prove that the files were never present in old commits. That was the reason the local commit history also had to be rebuilt.

12. Git Commands Used - What Each One Does

Command	Purpose
git status	Shows modified, staged, untracked files and branch state.
git diff	Displays unstaged content changes before committing.
git add .	Stages eligible changes for the next commit.
git commit -m "message"	Creates a local snapshot with a descriptive commit message.
git push	Sends local commits to the configured remote branch.
git push origin main	Explicitly pushes local main to the origin remote.
git remote -v	Shows the fetch/push remote URLs.
git config user.name	Shows the local Git commit author name.
git config user.email	Shows the local Git commit author email.
git ls-files	Lists files currently tracked by Git.
git rm --cached	Stops tracking a file while keeping it locally.
git rm -r --cached	Stops tracking a directory while keeping local contents.
git branch backup-before-cleanup	Creates a local backup pointer before history changes.
git reset --soft origin/main	Moves local HEAD to origin/main while preserving current changes staged.

13. Commit Messages Used During the Work

```
Fix camera stream URL handling
Fix camera streaming and exclude local FFmpeg binaries
Update camera streaming system
```

The exact wording of commit messages can vary. The important practice is to make the message describe the logical change rather than the shell commands used to make it.

14. Commit-Graph Warning

Git printed a warning such as **unable to find all commit-graph files**. This warning was separate from the GitHub large-file rejection. It did not identify the cause of the push failure; the push was rejected because the oversized files existed in the local history.

The practical debugging approach was to focus on the actual remote rejection message and then inspect tracked files and local commits.

15. What Belongs in This Repository

Keep / Commit	Do not commit
Python source code	.venv/
Dockerfiles	Windows FFmpeg .exe binaries
docker-compose.yml	Machine-specific executable folders
database/init.sql	Secrets and real camera passwords
MediaMTX YAML configuration	Generated runtime data / local databases
Frontend source	Local caches and temporary files

16. Final GitHub Workflow for Future Changes

```
# 1. Check branch and changed files
git status

# 2. Review changes
git diff
```

```
# 3. Check for accidentally tracked large/local files
git status
git ls-files | Select-String "ffmpeg|mediamtx.exe|.venv"

# 4. Stage intended changes
git add <specific-files>

# 5. Review staged changes
git diff --cached

# 6. Commit
git commit -m "Describe the change"

# 7. Push
git push origin main
```

Using specific files with **git add** can be safer than **git add .** when the repository contains local development tools.

17. Relationship to the Docker and MediaMTX Work

The GitHub repository should store the portable part of the Dockerized streaming architecture: Dockerfiles, Compose configuration, Python backend code, database schema and MediaMTX configuration. Local Windows tools such as FFmpeg and the Windows MediaMTX executable can remain on the developer machine or be supplied through a container/build process.

```
GitHub repository
|
+-- backend/
+-- database/
+-- deployment/
|   +-- docker-compose.yml
|   +-- docker/
|       +-- backend.Dockerfile
|       +-- mediamtx.Dockerfile
|
+-- streaming/
|   +-- configuration
|
+-- frontend/

Local machine only
|
+-- .venv/
+-- Windows FFmpeg binaries
+-- Windows MediaMTX executable
```

18. Security Note

During development a real RTSP credential was pasted into the conversation. It is intentionally not reproduced in this document. Real camera credentials should not be stored in public Git history. For a production deployment, keep credentials in environment variables or another secret-management mechanism and avoid committing them to GitHub.

19. Result of the Cleanup

The local Git history was rebuilt from the accepted origin/main commit so the unpushed history no longer depended on the oversized FFmpeg binaries. The local runtime files could remain on the developer machine because Git tracking and local disk storage are separate.

Core lesson: .gitignore prevents future tracking; it does not remove a file that already exists in old commits. When a large file has already entered unpushed history, the history itself must be cleaned.